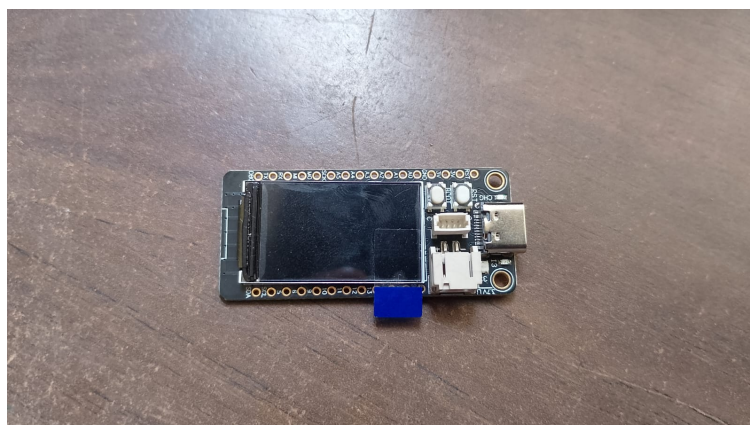

Entregable N.º 1

Semana 1 — Configuración del Entorno

Robótica Móvil con ESP32-S3 y Raspberry Pi



TENSTAR TS-ESP32S3

Autor: Camilo Callupe M.
Código: 20257036B
Fecha: 11 de mayo de 2026

Versión 1.0

Índice

1. Resumen del Entregable	3
2. Objetivos	3
2.1. Objetivo general	3
2.2. Objetivos específicos	3
3. Alcance del Entregable	3
4. Hardware Utilizado	4
5. Software y Herramientas Instaladas	5
6. Configuración del Entorno para ESP32-S3	5
6.1. IDE y framework utilizados	5
6.2. Parámetros de placa y comunicación	6
7. Configuración del Entorno para Raspberry Pi	6
7.1. Sistema operativo instalado	6
7.2. Comandos principales	6
8. Librerías y Dependencias	7
8.1. Dependencias instaladas para la Semana 1	7
8.2. Dependencias previstas para etapas posteriores	7
9. Prueba Básica de Funcionamiento	8
9.1. Procedimiento	8
9.2. Código mínimo de prueba	8
9.3. Resultado obtenido	11
10.Evidencias de Configuración	12
10.1. Entorno de desarrollo en Visual Studio Code	12
10.2. Salida del monitor serial	13
10.3. Raspberry Pi configurada	13
11.Problemas Encontrados y Soluciones Aplicadas	14
12.Verificación Final	14
13.Conclusiones	15
14.Recomendaciones	15
15.Referencias	16

Anexos

17

1. Resumen del Entregable

Descripción general

El presente documento registra la configuración inicial del entorno de desarrollo para el proyecto **Robótica Móvil con ESP32-S3 y Raspberry Pi**. Se documentan el hardware utilizado, las herramientas instaladas, las versiones principales de software, la configuración del ESP32-S3, la preparación de la Raspberry Pi y la prueba básica de comunicación mediante monitor serial.

El propósito del entregable es verificar que el entorno base se encuentra operativo para iniciar las siguientes actividades del proyecto: lectura de sensores, comunicación entre dispositivos, control de motores e integración progresiva del sistema.

2. Objetivos

2.1. Objetivo general

Configurar y verificar el entorno de desarrollo necesario para programar, cargar y monitorear aplicaciones básicas en el ESP32-S3, así como preparar la Raspberry Pi para tareas posteriores de procesamiento y comunicación.

2.2. Objetivos específicos

Instalar Visual Studio Code, ESP-IDF y las herramientas de compilación para ESP32-S3.

Registrar las versiones principales del sistema operativo, compilador, Python, Git y dependencias instaladas.

Verificar la detección del ESP32-S3 mediante el puerto serial `/dev/ttyACM0`.

Configurar Raspberry Pi OS en la Raspberry Pi 5 y habilitar herramientas básicas de trabajo.

Ejecutar una prueba *Hello World* en ESP-IDF y validar la salida mediante el monitor serial.

Documentar problemas encontrados y soluciones aplicadas durante la configuración.

3. Alcance del Entregable

Este entregable cubre únicamente la preparación del entorno de desarrollo y la validación básica del hardware principal. No incluye todavía lectura de sensores, control de motores,

comunicación UART entre ESP32-S3 y Raspberry Pi, integración de LiDAR, cámara ni navegación del robot.

4. Hardware Utilizado

#	Componente	Modelo / Referencia	Función en la Semana 1
1	Microcontrolador	TENSTAR TS-ESP32-S3	Verificación de conexión, carga de firmware y prueba inicial.
2	Computadora personal	Laptop / PC de trabajo	Instalación del IDE, compilador y herramientas de desarrollo.
3	Raspberry Pi	Raspberry Pi 5	Configuración del sistema operativo y entorno de alto nivel.
4	Cable USB	USB-C / micro-USB	Programación y comunicación serial con el ESP32-S3.
5	Tarjeta microSD	128 GB	Instalación del sistema operativo de la Raspberry Pi.
6	Fuente de alimentación	5 V / 3 A o superior	Alimentación estable de la Raspberry Pi.

Tabla 1: Hardware utilizado para la configuración inicial

5. Software y Herramientas Instaladas

Herramienta	Versión	Uso principal
Sistema operativo de la PC	Debian Testing (Forky)	Entorno principal de desarrollo.
Visual Studio Code	1.119.0	Edición, compilación y monitoreo del proyecto ESP-IDF.
ESP-IDF	6.0.1	Framework oficial de desarrollo para ESP32.
Python	3.12.13	Ejecución de herramientas auxiliares y scripts.
Git	2.53.0	Control de versiones del proyecto.
Raspberry Pi OS	Debian 13 (Trixie)	Sistema operativo instalado en la Raspberry Pi.
xtensa-esp32s3-elf-gcc	esp-15.2.0	Compilador utilizado para el firmware del ESP32-S3.

Tabla 2: Software y herramientas principales instaladas

6. Configuración del Entorno para ESP32-S3

6.1. IDE y framework utilizados

Para el desarrollo del firmware se utilizó **Visual Studio Code** junto con la extensión oficial de **ESP-IDF**. Esta configuración permite crear proyectos, seleccionar el target, compilar, flashear y monitorear la salida serial desde un único entorno de trabajo.

Elemento	Configuración
IDE utilizado	Visual Studio Code 1.119.0
Framework	ESP-IDF 6.0.1
Sistema operativo de trabajo	Debian Testing (Forky)
Método de instalación	Espresif Installation Manager y extensión ESP-IDF

Tabla 3: Entorno de desarrollo utilizado para ESP32-S3

6.2. Parámetros de placa y comunicación

Parámetro	Valor configurado
Target	ESP32S3
Placa	ESP32S3 Custom Board
Puerto serial	/dev/ttyACM0
Velocidad de carga	115200 baudios
Frecuencia de CPU	160 MHz
Modo USB	UART mediante USB-to-UART
Monitor serial	115200 baudios

Tabla 4: Configuración inicial del ESP32-S3

Verificación realizada

Se creó un proyecto de prueba en ESP-IDF, se seleccionó el target ESP32-S3, se configuró el puerto `/dev/ttyACM0`, se compiló el firmware y se cargó correctamente en el microcontrolador.

7. Configuración del Entorno para Raspberry Pi

7.1. Sistema operativo instalado

Elemento	Descripción
Modelo	Raspberry Pi 5
Sistema operativo	Raspberry Pi OS
Versión base	Debian 13 Trixie
Herramienta de instalación	Raspberry Pi Imager
Almacenamiento	microSD de 128 GB
Conexión de red	WiFi
Acceso remoto	SSH habilitado

Tabla 5: Configuración del sistema operativo en Raspberry Pi

7.2. Comandos principales

Listing 1: Actualización e instalación básica en Raspberry Pi

```

1 sudo apt update
2 sudo apt upgrade -y
3 sudo apt install python3 python3-pip git -y

```

8. Librerías y Dependencias

8.1. Dependencias instaladas para la Semana 1

Librería / Dependencia	Versión	Uso principal
pyserial	3.5	Comunicación serial entre Raspberry Pi y ESP32-S3.
gpiozero	2.0.1	Control básico de pines GPIO en Raspberry Pi.
RPi.GPIO	0.7.1	Manejo de entradas y salidas digitales.
numpy	2.4.4	Procesamiento numérico básico.
opencv-python	4.13.0.92	Procesamiento de imágenes en etapas posteriores.

Tabla 6: Dependencias instaladas o verificadas en el entorno Python

8.2. Dependencias previstas para etapas posteriores

Las siguientes dependencias no forman parte esencial de la Semana 1, pero fueron registradas porque podrían emplearse en etapas posteriores de visión artificial, prototipado o inferencia de modelos.

Dependencia	Versión	Uso previsto
TensorFlow	2.21.0	Pruebas futuras de inferencia o modelos de visión.
YOLO	0.3.2	Pruebas futuras de detección de objetos.

Tabla 7: Dependencias previstas para fases posteriores del proyecto

Para verificar las versiones instaladas se utilizaron los siguientes comandos:

Listing 2: Verificación de versiones en Python

```

1 python3 --version

```



```
2 pip3 --version
3 pip3 list
```

9. Prueba Básica de Funcionamiento

9.1. Procedimiento

La verificación inicial del ESP32-S3 se realizó mediante un proyecto básico de ESP-IDF. El procedimiento consistió en conectar la placa por USB, seleccionar el target ESP32-S3, compilar el ejemplo *Hello World*, cargarlo al microcontrolador y observar la salida mediante el monitor serial.

Listing 3: Comandos de compilación, carga y monitoreo

```
1 idf.py set-target esp32s3
2 idf.py build
3 idf.py -p /dev/ttyACM0 flash monitor
```

9.2. Código mínimo de prueba

En el cuerpo del informe se muestra solo el fragmento esencial del programa. El código completo se incluye en el Anexo A.

Listing 4: Fragmento principal del programa de prueba

```
1  /*
2   * SPDX-FileCopyrightText: 2010-2022 Espressif Systems (Shanghai) CO LTD
3   *
4   * SPDX-License-Identifier: CC0-1.0
5   */
6
7  #include <stdio.h>
8  #include <inttypes.h>
9  #include "sdkconfig.h"
10 #include "freertos/FreeRTOS.h"
11 #include "freertos/task.h"
12 #include "esp_chip_info.h"
13 #include "esp_flash.h"
14 #include "esp_system.h"
15
16 void app_main(void)
17 {
18     printf("Hello_world!\n");
19
20     /* Print chip information */
21 }
```

```

22 // Declaramos variables chip_info y flash_size. chip_info es una
    estructura y flash_size es un unsigned
23 // chip_info es una estructura con los siguientes atributos: model,
    features, revision y cores.
24 //flash_size almacenara el valor del tamao de memoria flash del esp32-s3
25 //Ambas variables estan declaradas pero no inicializadas aun, les
    daremos valores validos mediante funciones en las siguientes lineas
26 esp_chip_info_t chip_info;
27 uint32_t flash_size;
28
29
30 esp_chip_info(&chip_info); // Funcion que obtiene la informacion del
    chip(desde el mismo chip) y la guarda en el valor que almacena la
    direccion de memoria de chip_info
31 // modificando de esta manera el valor o
    valores de los atributos de la estructura
32
33
34 //Aca imprimimos la informacin del chip que tenemos. Tenemos dos
    placeholder al inicio y 4 al final
35 //Los primeros dos toman una variable de la configuracion del proyecto(
    el target) y la otra un atributo de la estructura llamada chip_info(
    numero de cores)
36 // Luego, se hacen condicionales para definir el valor de los ultimos 4
    placeholders
37 // Para cada uno, se hace una operacion AND bit a bit de cada valor de
    chip_info con las features,
38
39 printf("This is %s chip with %d CPU core(s), %s %s %s %s",
40        CONFIG_IDF_TARGET,
41        chip_info.cores,
42        (chip_info.features & CHIP_FEATURE_WIFI_BGN) ? "WiFi/" : "",
43        (chip_info.features & CHIP_FEATURE_BT) ? "BT" : "",
44        (chip_info.features & CHIP_FEATURE_BLE) ? "BLE" : "",
45        (chip_info.features & CHIP_FEATURE_IEEE802154) ? ", 802.15.4 ("
        Zigbee/Thread)" : "");
46
47
48 // Aca declaramos dos variables de tipo unsigned, major_rev y minor_rev
49 // La revisin del silicio identifica la versin fsica del chip.
50 // chip_info.revision contiene esa revisin codificada como nmero entero.
51 // Se separa en parte mayor y menor:
52 // - major_rev obtiene la parte entera principal.
53 // - minor_rev obtiene la parte secundaria.
54 unsigned major_rev = chip_info.revision / 100;
55 unsigned minor_rev = chip_info.revision % 100;

```

```
56
57
58 //Usamos las variables anteriormente definidas y las usamos para
    imprimir la version del chip
59 printf("silicon_revision_v%d.%d,\n", major_rev, minor_rev);
60
61 // Obtiene el tamaño de la memoria flash principal.
62 // NULL indica que se usará la flash por defecto del sistema.
63 // &flash_size pasa la dirección de memoria de flash_size,
64 // porque la función escribir allí el tamaño obtenido.
65 // ESP_OK representa operación exitosa.
66 // Si la función devuelve algo diferente de ESP_OK, ocurrió un error.
67 if(esp_flash_get_size(NULL, &flash_size) != ESP_OK) {
68     printf("Get_flash_size_failed");
69     return;
70 }
71
72
73 // Tenemos una impresión con dos placeholders.
74 //El primero es el resultado de flash_size / (uint32_t)(1024*1024), lo
    que nos da el espacio flash en MB
75 // el segundo es una condicional, si (chip_info.features contiene la
    feature CHIP_FEATURE_EMB_FLASH) entonces será igual a "embedded",
    sino "external"
76 //esto último nos dice si el almacenamiento está embebido en el chip o
    es externo. Esto debido a que el esp32 admite almacenamiento externo
77
78 printf("%" PRIu32 "MB %sflash\n", flash_size / (uint32_t)(1024 * 1024),
79
    (chip_info.features & CHIP_FEATURE_EMB_FLASH) ? "embedded" : "
        external");
80
81 //Imprime el minimum free heap size
82 //el minimum free heap size es una métrica que indica la menor cantidad
    de heap libre
83 //que ha existido desde que el sistema ha iniciado
84 //el heap de memoria es una región de memoria dinámica utilizada por los
    programas
85 //para almacenar datos cuyo tamaño o tiempo de vida no se conocen de
    antemano
86
87 printf("Minimum_free_heap_size: %" PRIu32 " bytes\n",
    esp_get_minimum_free_heap_size());
88
89 //Ciclo for
90 //Contador que empieza en 10 y baja 1 cada ciclo
```

```
91 //Por cada ciclo imprime Restarting in %d seconds, siendo d=i
92 // portTICK_PERIOD_MS es una variable de freeRTOS
93 // Se usa esta variable para introducir un delay en cada ciclo.
94 // En FreeRTOS, port se refiere a la adaptacin del sistema operativo a
    una arquitectura/plataforma.
95 // portTICK_PERIOD_MS representa cuntos milisegundos dura un tick del
    sistema.
96 for (int i = 10; i >= 0; i--) {
97     printf("Restarting in %d seconds...\n", i);
98     vTaskDelay(1000 / portTICK_PERIOD_MS);
99 }
100
101 //Imprimimos que se esta reiniciando
102 printf("Restarting now.\n");
103
104
105 //stdout es el flujo de salida estndar, una macro que representa un
    puntero a un archivo preabierto
106 // // Fuerza el envo inmediato de cualquier texto pendiente en stdout.
107 // Esto evita que el mensaje anterior se pierda antes del reinicio.
108 fflush(stdout);
109 //reinicia el esp
110 esp_restart();
111
112
113 // Todo lo impreso con printf se enva a stdout.
114 // En ESP-IDF, stdout normalmente est conectado a la consola del sistema.
115
116 // Esa consola se observa desde la PC mediante el monitor serial,
117 // por ejemplo con idf.py monitor o desde la extensin de ESP-IDF en VS
    Code.
118 // En ESP32-S3, la consola puede salir por UART o por USB CDC,
119 // dependiendo de la placa y de la configuracin del proyecto.
120 }
```

9.3. Resultado obtenido

Resultado de la prueba

El firmware fue compilado y cargado correctamente en el ESP32-S3. Al abrir el monitor serial a 115200 baudios, se observó el mensaje **Hello world!**, la identificación del chip, la memoria flash detectada y el valor de memoria heap libre. Esto confirma la comunicación funcional entre la computadora y el microcontrolador.

10. Evidencias de Configuración

Las siguientes capturas documentan la configuración del entorno, la ejecución del monitor serial y el estado inicial de la Raspberry Pi.

10.1. Entorno de desarrollo en Visual Studio Code



Figura 1: Visual Studio Code configurado con ESP-IDF para ESP32-S3

10.2. Salida del monitor serial

```

main > C:\hello_world_main.c X  # sdcconfig
main > C:\hello_world_main.c > app_main(void)
16 void app_main(void)
17 {
18     esp_chip_info(chip_info); // Funcion que obtiene la informacion del chip(desde el mismo chip) y la guarda en el
19     // modificando de esta manera el valor o valores de los atributos de la estructura
20
21     // Aca imprimimos la informacion del chip que tenemos, Tenemos dos placeholder al inicio y 4 al final
22     // Los primeros dos toman una variable de la configuracion del proyecto(el target) y la otra un atributo de la estructura llamada chip_info(numero de cores)
23     // Luego, se hacen condicionales para definir el valor de los ultimos 4 placeholders
24     // Para cada uno, se hace una operacion AND bit a bit de cada valor de chip_info con las features.
25
26     printf("This is %s chip with %d CPU core(s), %s%s%s%s, ",
27           CONFIG_IDF_TARGET,
28           chip_info.cores,
29           (chip_info.features & CHIP_FEATURE_WIFI_BGN) ? "WiFi/" : "",
30           (chip_info.features & CHIP_FEATURE_BT) ? "BT" : "",
31           (chip_info.features & CHIP_FEATURE_BLE) ? "BLE" : "",
32           (chip_info.features & CHIP_FEATURE_IEEE802154) ? ", 802.15.4 (Zigbee/Thread)" : "");
33
34     // Aca declaramos dos variables de tipo unsigned, major_rev y minor_rev
35     // La revision del silicio identifica la version fisica del chip.
36     // chip_info.revision contiene esa revision codificada como numero entero.
37     // Se separa en parte mayor y menor.
38     // - major_rev obtiene la parte entera principal.
39     // - minor_rev obtiene la parte secundaria.
40     unsigned major_rev = chip_info.revision / 100;
41     unsigned minor_rev = chip_info.revision % 100;
42
43     // Usamos las variables anteriormente definidas y las usamos para imprimir la version del chip
44     printf("silicon revision v%d.%d, ", major_rev, minor_rev);
45
46     // Obtiene el tamaño de la memoria flash principal.
47
48     printf("Minimum free heap size: %d bytes\n", heap_caps_get_free_size(MALLOC_CAP_8BIT));
49 }
50
51 // Obtener el tamaño de la memoria flash principal.
52 void get_flash_size()
53 {
54     uint32_t flash_size = 0;
55     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
56     if (err == ESP_OK)
57     {
58         printf("Flash size: %d bytes\n", flash_size);
59     }
60 }
61
62 int main()
63 {
64     printf("Hello world!\n");
65     get_flash_size();
66     return 0;
67 }
68
69 // Obtener el tamaño de la memoria flash principal.
70 void get_flash_size()
71 {
72     uint32_t flash_size = 0;
73     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
74     if (err == ESP_OK)
75     {
76         printf("Flash size: %d bytes\n", flash_size);
77     }
78 }
79
80 int main()
81 {
82     printf("Hello world!\n");
83     get_flash_size();
84     return 0;
85 }
86
87 // Obtener el tamaño de la memoria flash principal.
88 void get_flash_size()
89 {
90     uint32_t flash_size = 0;
91     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
92     if (err == ESP_OK)
93     {
94         printf("Flash size: %d bytes\n", flash_size);
95     }
96 }
97
98 int main()
99 {
100    printf("Hello world!\n");
101    get_flash_size();
102    return 0;
103 }
104
105 // Obtener el tamaño de la memoria flash principal.
106 void get_flash_size()
107 {
108     uint32_t flash_size = 0;
109     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
110     if (err == ESP_OK)
111     {
112         printf("Flash size: %d bytes\n", flash_size);
113     }
114 }
115
116 int main()
117 {
118     printf("Hello world!\n");
119     get_flash_size();
120     return 0;
121 }
122
123 // Obtener el tamaño de la memoria flash principal.
124 void get_flash_size()
125 {
126     uint32_t flash_size = 0;
127     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
128     if (err == ESP_OK)
129     {
130         printf("Flash size: %d bytes\n", flash_size);
131     }
132 }
133
134 int main()
135 {
136     printf("Hello world!\n");
137     get_flash_size();
138     return 0;
139 }
140
141 // Obtener el tamaño de la memoria flash principal.
142 void get_flash_size()
143 {
144     uint32_t flash_size = 0;
145     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
146     if (err == ESP_OK)
147     {
148         printf("Flash size: %d bytes\n", flash_size);
149     }
150 }
151
152 int main()
153 {
154     printf("Hello world!\n");
155     get_flash_size();
156     return 0;
157 }
158
159 // Obtener el tamaño de la memoria flash principal.
160 void get_flash_size()
161 {
162     uint32_t flash_size = 0;
163     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
164     if (err == ESP_OK)
165     {
166         printf("Flash size: %d bytes\n", flash_size);
167     }
168 }
169
170 int main()
171 {
172     printf("Hello world!\n");
173     get_flash_size();
174     return 0;
175 }
176
177 // Obtener el tamaño de la memoria flash principal.
178 void get_flash_size()
179 {
180     uint32_t flash_size = 0;
181     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
182     if (err == ESP_OK)
183     {
184         printf("Flash size: %d bytes\n", flash_size);
185     }
186 }
187
188 int main()
189 {
190     printf("Hello world!\n");
191     get_flash_size();
192     return 0;
193 }
194
195 // Obtener el tamaño de la memoria flash principal.
196 void get_flash_size()
197 {
198     uint32_t flash_size = 0;
199     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
200     if (err == ESP_OK)
201     {
202         printf("Flash size: %d bytes\n", flash_size);
203     }
204 }
205
206 int main()
207 {
208     printf("Hello world!\n");
209     get_flash_size();
210     return 0;
211 }
212
213 // Obtener el tamaño de la memoria flash principal.
214 void get_flash_size()
215 {
216     uint32_t flash_size = 0;
217     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
218     if (err == ESP_OK)
219     {
220         printf("Flash size: %d bytes\n", flash_size);
221     }
222 }
223
224 int main()
225 {
226     printf("Hello world!\n");
227     get_flash_size();
228     return 0;
229 }
230
231 // Obtener el tamaño de la memoria flash principal.
232 void get_flash_size()
233 {
234     uint32_t flash_size = 0;
235     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
236     if (err == ESP_OK)
237     {
238         printf("Flash size: %d bytes\n", flash_size);
239     }
240 }
241
242 int main()
243 {
244     printf("Hello world!\n");
245     get_flash_size();
246     return 0;
247 }
248
249 // Obtener el tamaño de la memoria flash principal.
250 void get_flash_size()
251 {
252     uint32_t flash_size = 0;
253     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
254     if (err == ESP_OK)
255     {
256         printf("Flash size: %d bytes\n", flash_size);
257     }
258 }
259
260 int main()
261 {
262     printf("Hello world!\n");
263     get_flash_size();
264     return 0;
265 }
266
267 // Obtener el tamaño de la memoria flash principal.
268 void get_flash_size()
269 {
270     uint32_t flash_size = 0;
271     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
272     if (err == ESP_OK)
273     {
274         printf("Flash size: %d bytes\n", flash_size);
275     }
276 }
277
278 int main()
279 {
280     printf("Hello world!\n");
281     get_flash_size();
282     return 0;
283 }
284
285 // Obtener el tamaño de la memoria flash principal.
286 void get_flash_size()
287 {
288     uint32_t flash_size = 0;
289     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
290     if (err == ESP_OK)
291     {
292         printf("Flash size: %d bytes\n", flash_size);
293     }
294 }
295
296 int main()
297 {
298     printf("Hello world!\n");
299     get_flash_size();
300     return 0;
301 }
302
303 // Obtener el tamaño de la memoria flash principal.
304 void get_flash_size()
305 {
306     uint32_t flash_size = 0;
307     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
308     if (err == ESP_OK)
309     {
310         printf("Flash size: %d bytes\n", flash_size);
311     }
312 }
313
314 int main()
315 {
316     printf("Hello world!\n");
317     get_flash_size();
318     return 0;
319 }
320
321 // Obtener el tamaño de la memoria flash principal.
322 void get_flash_size()
323 {
324     uint32_t flash_size = 0;
325     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
326     if (err == ESP_OK)
327     {
328         printf("Flash size: %d bytes\n", flash_size);
329     }
330 }
331
332 int main()
333 {
334     printf("Hello world!\n");
335     get_flash_size();
336     return 0;
337 }
338
339 // Obtener el tamaño de la memoria flash principal.
340 void get_flash_size()
341 {
342     uint32_t flash_size = 0;
343     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
344     if (err == ESP_OK)
345     {
346         printf("Flash size: %d bytes\n", flash_size);
347     }
348 }
349
350 int main()
351 {
352     printf("Hello world!\n");
353     get_flash_size();
354     return 0;
355 }
356
357 // Obtener el tamaño de la memoria flash principal.
358 void get_flash_size()
359 {
360     uint32_t flash_size = 0;
361     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
362     if (err == ESP_OK)
363     {
364         printf("Flash size: %d bytes\n", flash_size);
365     }
366 }
367
368 int main()
369 {
370     printf("Hello world!\n");
371     get_flash_size();
372     return 0;
373 }
374
375 // Obtener el tamaño de la memoria flash principal.
376 void get_flash_size()
377 {
378     uint32_t flash_size = 0;
379     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
380     if (err == ESP_OK)
381     {
382         printf("Flash size: %d bytes\n", flash_size);
383     }
384 }
385
386 int main()
387 {
388     printf("Hello world!\n");
389     get_flash_size();
390     return 0;
391 }
392
393 // Obtener el tamaño de la memoria flash principal.
394 void get_flash_size()
395 {
396     uint32_t flash_size = 0;
397     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
398     if (err == ESP_OK)
399     {
400         printf("Flash size: %d bytes\n", flash_size);
401     }
402 }
403
404 int main()
405 {
406     printf("Hello world!\n");
407     get_flash_size();
408     return 0;
409 }
410
411 // Obtener el tamaño de la memoria flash principal.
412 void get_flash_size()
413 {
414     uint32_t flash_size = 0;
415     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
416     if (err == ESP_OK)
417     {
418         printf("Flash size: %d bytes\n", flash_size);
419     }
420 }
421
422 int main()
423 {
424     printf("Hello world!\n");
425     get_flash_size();
426     return 0;
427 }
428
429 // Obtener el tamaño de la memoria flash principal.
430 void get_flash_size()
431 {
432     uint32_t flash_size = 0;
433     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
434     if (err == ESP_OK)
435     {
436         printf("Flash size: %d bytes\n", flash_size);
437     }
438 }
439
440 int main()
441 {
442     printf("Hello world!\n");
443     get_flash_size();
444     return 0;
445 }
446
447 // Obtener el tamaño de la memoria flash principal.
448 void get_flash_size()
449 {
450     uint32_t flash_size = 0;
451     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
452     if (err == ESP_OK)
453     {
454         printf("Flash size: %d bytes\n", flash_size);
455     }
456 }
457
458 int main()
459 {
460     printf("Hello world!\n");
461     get_flash_size();
462     return 0;
463 }
464
465 // Obtener el tamaño de la memoria flash principal.
466 void get_flash_size()
467 {
468     uint32_t flash_size = 0;
469     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
470     if (err == ESP_OK)
471     {
472         printf("Flash size: %d bytes\n", flash_size);
473     }
474 }
475
476 int main()
477 {
478     printf("Hello world!\n");
479     get_flash_size();
480     return 0;
481 }
482
483 // Obtener el tamaño de la memoria flash principal.
484 void get_flash_size()
485 {
486     uint32_t flash_size = 0;
487     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
488     if (err == ESP_OK)
489     {
490         printf("Flash size: %d bytes\n", flash_size);
491     }
492 }
493
494 int main()
495 {
496     printf("Hello world!\n");
497     get_flash_size();
498     return 0;
499 }
500
501 // Obtener el tamaño de la memoria flash principal.
502 void get_flash_size()
503 {
504     uint32_t flash_size = 0;
505     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
506     if (err == ESP_OK)
507     {
508         printf("Flash size: %d bytes\n", flash_size);
509     }
510 }
511
512 int main()
513 {
514     printf("Hello world!\n");
515     get_flash_size();
516     return 0;
517 }
518
519 // Obtener el tamaño de la memoria flash principal.
520 void get_flash_size()
521 {
522     uint32_t flash_size = 0;
523     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
524     if (err == ESP_OK)
525     {
526         printf("Flash size: %d bytes\n", flash_size);
527     }
528 }
529
530 int main()
531 {
532     printf("Hello world!\n");
533     get_flash_size();
534     return 0;
535 }
536
537 // Obtener el tamaño de la memoria flash principal.
538 void get_flash_size()
539 {
540     uint32_t flash_size = 0;
541     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
542     if (err == ESP_OK)
543     {
544         printf("Flash size: %d bytes\n", flash_size);
545     }
546 }
547
548 int main()
549 {
550     printf("Hello world!\n");
551     get_flash_size();
552     return 0;
553 }
554
555 // Obtener el tamaño de la memoria flash principal.
556 void get_flash_size()
557 {
558     uint32_t flash_size = 0;
559     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
560     if (err == ESP_OK)
561     {
562         printf("Flash size: %d bytes\n", flash_size);
563     }
564 }
565
566 int main()
567 {
568     printf("Hello world!\n");
569     get_flash_size();
570     return 0;
571 }
572
573 // Obtener el tamaño de la memoria flash principal.
574 void get_flash_size()
575 {
576     uint32_t flash_size = 0;
577     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
578     if (err == ESP_OK)
579     {
580         printf("Flash size: %d bytes\n", flash_size);
581     }
582 }
583
584 int main()
585 {
586     printf("Hello world!\n");
587     get_flash_size();
588     return 0;
589 }
590
591 // Obtener el tamaño de la memoria flash principal.
592 void get_flash_size()
593 {
594     uint32_t flash_size = 0;
595     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
596     if (err == ESP_OK)
597     {
598         printf("Flash size: %d bytes\n", flash_size);
599     }
600 }
601
602 int main()
603 {
604     printf("Hello world!\n");
605     get_flash_size();
606     return 0;
607 }
608
609 // Obtener el tamaño de la memoria flash principal.
610 void get_flash_size()
611 {
612     uint32_t flash_size = 0;
613     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
614     if (err == ESP_OK)
615     {
616         printf("Flash size: %d bytes\n", flash_size);
617     }
618 }
619
620 int main()
621 {
622     printf("Hello world!\n");
623     get_flash_size();
624     return 0;
625 }
626
627 // Obtener el tamaño de la memoria flash principal.
628 void get_flash_size()
629 {
630     uint32_t flash_size = 0;
631     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
632     if (err == ESP_OK)
633     {
634         printf("Flash size: %d bytes\n", flash_size);
635     }
636 }
637
638 int main()
639 {
640     printf("Hello world!\n");
641     get_flash_size();
642     return 0;
643 }
644
645 // Obtener el tamaño de la memoria flash principal.
646 void get_flash_size()
647 {
648     uint32_t flash_size = 0;
649     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
650     if (err == ESP_OK)
651     {
652         printf("Flash size: %d bytes\n", flash_size);
653     }
654 }
655
656 int main()
657 {
658     printf("Hello world!\n");
659     get_flash_size();
660     return 0;
661 }
662
663 // Obtener el tamaño de la memoria flash principal.
664 void get_flash_size()
665 {
666     uint32_t flash_size = 0;
667     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
668     if (err == ESP_OK)
669     {
670         printf("Flash size: %d bytes\n", flash_size);
671     }
672 }
673
674 int main()
675 {
676     printf("Hello world!\n");
677     get_flash_size();
678     return 0;
679 }
680
681 // Obtener el tamaño de la memoria flash principal.
682 void get_flash_size()
683 {
684     uint32_t flash_size = 0;
685     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
686     if (err == ESP_OK)
687     {
688         printf("Flash size: %d bytes\n", flash_size);
689     }
690 }
691
692 int main()
693 {
694     printf("Hello world!\n");
695     get_flash_size();
696     return 0;
697 }
698
699 // Obtener el tamaño de la memoria flash principal.
700 void get_flash_size()
701 {
702     uint32_t flash_size = 0;
703     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
704     if (err == ESP_OK)
705     {
706         printf("Flash size: %d bytes\n", flash_size);
707     }
708 }
709
710 int main()
711 {
712     printf("Hello world!\n");
713     get_flash_size();
714     return 0;
715 }
716
717 // Obtener el tamaño de la memoria flash principal.
718 void get_flash_size()
719 {
720     uint32_t flash_size = 0;
721     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
722     if (err == ESP_OK)
723     {
724         printf("Flash size: %d bytes\n", flash_size);
725     }
726 }
727
728 int main()
729 {
730     printf("Hello world!\n");
731     get_flash_size();
732     return 0;
733 }
734
735 // Obtener el tamaño de la memoria flash principal.
736 void get_flash_size()
737 {
738     uint32_t flash_size = 0;
739     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
740     if (err == ESP_OK)
741     {
742         printf("Flash size: %d bytes\n", flash_size);
743     }
744 }
745
746 int main()
747 {
748     printf("Hello world!\n");
749     get_flash_size();
750     return 0;
751 }
752
753 // Obtener el tamaño de la memoria flash principal.
754 void get_flash_size()
755 {
756     uint32_t flash_size = 0;
757     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
758     if (err == ESP_OK)
759     {
760         printf("Flash size: %d bytes\n", flash_size);
761     }
762 }
763
764 int main()
765 {
766     printf("Hello world!\n");
767     get_flash_size();
768     return 0;
769 }
770
771 // Obtener el tamaño de la memoria flash principal.
772 void get_flash_size()
773 {
774     uint32_t flash_size = 0;
775     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
776     if (err == ESP_OK)
777     {
778         printf("Flash size: %d bytes\n", flash_size);
779     }
780 }
781
782 int main()
783 {
784     printf("Hello world!\n");
785     get_flash_size();
786     return 0;
787 }
788
789 // Obtener el tamaño de la memoria flash principal.
790 void get_flash_size()
791 {
792     uint32_t flash_size = 0;
793     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
794     if (err == ESP_OK)
795     {
796         printf("Flash size: %d bytes\n", flash_size);
797     }
798 }
799
800 int main()
801 {
802     printf("Hello world!\n");
803     get_flash_size();
804     return 0;
805 }
806
807 // Obtener el tamaño de la memoria flash principal.
808 void get_flash_size()
809 {
810     uint32_t flash_size = 0;
811     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
812     if (err == ESP_OK)
813     {
814         printf("Flash size: %d bytes\n", flash_size);
815     }
816 }
817
818 int main()
819 {
820     printf("Hello world!\n");
821     get_flash_size();
822     return 0;
823 }
824
825 // Obtener el tamaño de la memoria flash principal.
826 void get_flash_size()
827 {
828     uint32_t flash_size = 0;
829     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
830     if (err == ESP_OK)
831     {
832         printf("Flash size: %d bytes\n", flash_size);
833     }
834 }
835
836 int main()
837 {
838     printf("Hello world!\n");
839     get_flash_size();
840     return 0;
841 }
842
843 // Obtener el tamaño de la memoria flash principal.
844 void get_flash_size()
845 {
846     uint32_t flash_size = 0;
847     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
848     if (err == ESP_OK)
849     {
850         printf("Flash size: %d bytes\n", flash_size);
851     }
852 }
853
854 int main()
855 {
856     printf("Hello world!\n");
857     get_flash_size();
858     return 0;
859 }
860
861 // Obtener el tamaño de la memoria flash principal.
862 void get_flash_size()
863 {
864     uint32_t flash_size = 0;
865     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
866     if (err == ESP_OK)
867     {
868         printf("Flash size: %d bytes\n", flash_size);
869     }
870 }
871
872 int main()
873 {
874     printf("Hello world!\n");
875     get_flash_size();
876     return 0;
877 }
878
879 // Obtener el tamaño de la memoria flash principal.
880 void get_flash_size()
881 {
882     uint32_t flash_size = 0;
883     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
884     if (err == ESP_OK)
885     {
886         printf("Flash size: %d bytes\n", flash_size);
887     }
888 }
889
890 int main()
891 {
892     printf("Hello world!\n");
893     get_flash_size();
894     return 0;
895 }
896
897 // Obtener el tamaño de la memoria flash principal.
898 void get_flash_size()
899 {
900     uint32_t flash_size = 0;
901     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
902     if (err == ESP_OK)
903     {
904         printf("Flash size: %d bytes\n", flash_size);
905     }
906 }
907
908 int main()
909 {
910     printf("Hello world!\n");
911     get_flash_size();
912     return 0;
913 }
914
915 // Obtener el tamaño de la memoria flash principal.
916 void get_flash_size()
917 {
918     uint32_t flash_size = 0;
919     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
920     if (err == ESP_OK)
921     {
922         printf("Flash size: %d bytes\n", flash_size);
923     }
924 }
925
926 int main()
927 {
928     printf("Hello world!\n");
929     get_flash_size();
930     return 0;
931 }
932
933 // Obtener el tamaño de la memoria flash principal.
934 void get_flash_size()
935 {
936     uint32_t flash_size = 0;
937     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
938     if (err == ESP_OK)
939     {
940         printf("Flash size: %d bytes\n", flash_size);
941     }
942 }
943
944 int main()
945 {
946     printf("Hello world!\n");
947     get_flash_size();
948     return 0;
949 }
950
951 // Obtener el tamaño de la memoria flash principal.
952 void get_flash_size()
953 {
954     uint32_t flash_size = 0;
955     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
956     if (err == ESP_OK)
957     {
958         printf("Flash size: %d bytes\n", flash_size);
959     }
960 }
961
962 int main()
963 {
964     printf("Hello world!\n");
965     get_flash_size();
966     return 0;
967 }
968
969 // Obtener el tamaño de la memoria flash principal.
970 void get_flash_size()
971 {
972     uint32_t flash_size = 0;
973     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
974     if (err == ESP_OK)
975     {
976         printf("Flash size: %d bytes\n", flash_size);
977     }
978 }
979
980 int main()
981 {
982     printf("Hello world!\n");
983     get_flash_size();
984     return 0;
985 }
986
987 // Obtener el tamaño de la memoria flash principal.
988 void get_flash_size()
989 {
990     uint32_t flash_size = 0;
991     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
992     if (err == ESP_OK)
993     {
994         printf("Flash size: %d bytes\n", flash_size);
995     }
996 }
997
998 int main()
999 {
1000    printf("Hello world!\n");
1001    get_flash_size();
1002    return 0;
1003 }
1004
1005 // Obtener el tamaño de la memoria flash principal.
1006 void get_flash_size()
1007 {
1008     uint32_t flash_size = 0;
1009     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
1010     if (err == ESP_OK)
1011     {
1012         printf("Flash size: %d bytes\n", flash_size);
1013     }
1014 }
1015
1016 int main()
1017 {
1018     printf("Hello world!\n");
1019     get_flash_size();
1020     return 0;
1021 }
1022
1023 // Obtener el tamaño de la memoria flash principal.
1024 void get_flash_size()
1025 {
1026     uint32_t flash_size = 0;
1027     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
1028     if (err == ESP_OK)
1029     {
1030         printf("Flash size: %d bytes\n", flash_size);
1031     }
1032 }
1033
1034 int main()
1035 {
1036     printf("Hello world!\n");
1037     get_flash_size();
1038     return 0;
1039 }
1040
1041 // Obtener el tamaño de la memoria flash principal.
1042 void get_flash_size()
1043 {
1044     uint32_t flash_size = 0;
1045     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
1046     if (err == ESP_OK)
1047     {
1048         printf("Flash size: %d bytes\n", flash_size);
1049     }
1050 }
1051
1052 int main()
1053 {
1054     printf("Hello world!\n");
1055     get_flash_size();
1056     return 0;
1057 }
1058
1059 // Obtener el tamaño de la memoria flash principal.
1060 void get_flash_size()
1061 {
1062     uint32_t flash_size = 0;
1063     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
1064     if (err == ESP_OK)
1065     {
1066         printf("Flash size: %d bytes\n", flash_size);
1067     }
1068 }
1069
1070 int main()
1071 {
1072     printf("Hello world!\n");
1073     get_flash_size();
1074     return 0;
1075 }
1076
1077 // Obtener el tamaño de la memoria flash principal.
1078 void get_flash_size()
1079 {
1080     uint32_t flash_size = 0;
1081     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
1082     if (err == ESP_OK)
1083     {
1084         printf("Flash size: %d bytes\n", flash_size);
1085     }
1086 }
1087
1088 int main()
1089 {
1090     printf("Hello world!\n");
1091     get_flash_size();
1092     return 0;
1093 }
1094
1095 // Obtener el tamaño de la memoria flash principal.
1096 void get_flash_size()
1097 {
1098     uint32_t flash_size = 0;
1099     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
1100     if (err == ESP_OK)
1101     {
1102         printf("Flash size: %d bytes\n", flash_size);
1103     }
1104 }
1105
1106 int main()
1107 {
1108     printf("Hello world!\n");
1109     get_flash_size();
1110     return 0;
1111 }
1112
1113 // Obtener el tamaño de la memoria flash principal.
1114 void get_flash_size()
1115 {
1116     uint32_t flash_size = 0;
1117     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
1118     if (err == ESP_OK)
1119     {
1120         printf("Flash size: %d bytes\n", flash_size);
1121     }
1122 }
1123
1124 int main()
1125 {
1126     printf("Hello world!\n");
1127     get_flash_size();
1128     return 0;
1129 }
1130
1131 // Obtener el tamaño de la memoria flash principal.
1132 void get_flash_size()
1133 {
1134     uint32_t flash_size = 0;
1135     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
1136     if (err == ESP_OK)
1137     {
1138         printf("Flash size: %d bytes\n", flash_size);
1139     }
1140 }
1141
1142 int main()
1143 {
1144     printf("Hello world!\n");
1145     get_flash_size();
1146     return 0;
1147 }
1148
1149 // Obtener el tamaño de la memoria flash principal.
1150 void get_flash_size()
1151 {
1152     uint32_t flash_size = 0;
1153     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
1154     if (err == ESP_OK)
1155     {
1156         printf("Flash size: %d bytes\n", flash_size);
1157     }
1158 }
1159
1160 int main()
1161 {
1162     printf("Hello world!\n");
1163     get_flash_size();
1164     return 0;
1165 }
1166
1167 // Obtener el tamaño de la memoria flash principal.
1168 void get_flash_size()
1169 {
1170     uint32_t flash_size = 0;
1171     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
1172     if (err == ESP_OK)
1173     {
1174         printf("Flash size: %d bytes\n", flash_size);
1175     }
1176 }
1177
1178 int main()
1179 {
1180     printf("Hello world!\n");
1181     get_flash_size();
1182     return 0;
1183 }
1184
1185 // Obtener el tamaño de la memoria flash principal.
1186 void get_flash_size()
1187 {
1188     uint32_t flash_size = 0;
1189     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
1190     if (err == ESP_OK)
1191     {
1192         printf("Flash size: %d bytes\n", flash_size);
1193     }
1194 }
1195
1196 int main()
1197 {
1198     printf("Hello world!\n");
1199     get_flash_size();
1200     return 0;
1201 }
1202
1203 // Obtener el tamaño de la memoria flash principal.
1204 void get_flash_size()
1205 {
1206     uint32_t flash_size = 0;
1207     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
1208     if (err == ESP_OK)
1209     {
1210         printf("Flash size: %d bytes\n", flash_size);
1211     }
1212 }
1213
1214 int main()
1215 {
1216     printf("Hello world!\n");
1217     get_flash_size();
1218     return 0;
1219 }
1220
1221 // Obtener el tamaño de la memoria flash principal.
1222 void get_flash_size()
1223 {
1224     uint32_t flash_size = 0;
1225     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
1226     if (err == ESP_OK)
1227     {
1228         printf("Flash size: %d bytes\n", flash_size);
1229     }
1230 }
1231
1232 int main()
1233 {
1234     printf("Hello world!\n");
1235     get_flash_size();
1236     return 0;
1237 }
1238
1239 // Obtener el tamaño de la memoria flash principal.
1240 void get_flash_size()
1241 {
1242     uint32_t flash_size = 0;
1243     esp_err_t err = esp_flash_get_size(NULL, &flash_size);
1244     if (err == ESP_OK)
1245     {
1246         printf("Flash size: %d bytes\n", flash_size);
1247     }
1248 }
1249
1250 int main()
1251 {
1252     printf("Hello world!\n");
1253     get_flash_size();
1254     return 0;
1255 }
1256
1257 // Obtener el tamaño de la memoria flash principal.
1258 void get_flash_size()
1259 {
1260     uint32_t flash_size = 0;
1261     esp_err_t err
```

Problema	Causa probable	Solución aplicada
ESP-IDF no detectó automáticamente el puerto del ESP32-S3	El usuario no tenía permisos sobre el grupo <code>dialout</code> .	Se agregó el usuario al grupo <code>dialout</code> y se reinició la sesión.
Fallo al flashear el microcontrolador	La placa no ingresó correctamente en modo descarga.	Se colocó el ESP32-S3 en modo <i>Download</i> antes de cargar el firmware.
Raspberry Pi sin conexión a red	Configuración WiFi incorrecta.	Se revisaron el SSID, la contraseña y los parámetros de conexión inicial.

Tabla 8: Problemas encontrados durante la configuración inicial

Comando utilizado para corregir permisos de puerto serial:

Listing 5: Asignación de permisos al grupo `dialout`

```
1 sudo usermod -aG dialout $USER
```

12. Verificación Final

Estado final del entorno

Al finalizar la Semana 1, el entorno de desarrollo quedó operativo. Se verificó la instalación de ESP-IDF, la detección del ESP32-S3, la carga del firmware de prueba, la lectura del monitor serial y la configuración básica de la Raspberry Pi 5.

Elemento verificado	Estado
Sistema operativo de la computadora identificado	Correcto
Visual Studio Code instalado	Correcto
ESP-IDF instalado y configurado	Correcto
Target ESP32-S3 seleccionado	Correcto
Puerto <code>/dev/ttyACM0</code> detectado	Correcto
Firmware de prueba compilado	Correcto
Firmware cargado al ESP32-S3	Correcto
Monitor serial funcionando	Correcto
Raspberry Pi OS instalado	Correcto
Python, Git y dependencias básicas verificadas	Correcto

Tabla 9: Resumen de verificación final

13. Conclusiones

Se configuró correctamente el entorno de desarrollo para trabajar con el ESP32-S3 mediante Visual Studio Code y ESP-IDF. La prueba *Hello World* permitió validar la compilación, carga y monitoreo serial del microcontrolador.

Asimismo, se preparó la Raspberry Pi 5 con Raspberry Pi OS, conexión de red, SSH y herramientas básicas de desarrollo. Con ello, el entorno queda listo para iniciar las siguientes etapas del proyecto, orientadas a sensores, comunicación y control del robot.

14. Recomendaciones

Verificar siempre el target, la placa y el puerto serial antes de compilar o flashear.

Mantener un registro de versiones de ESP-IDF, Python, Git y dependencias críticas.

Usar cables USB de datos y no solo de carga para evitar fallos de detección.

15. Referencias

Referencias

- [1] Espressif Systems. *ESP32-S3 Technical Reference Manual*. Disponible en: <https://www.espressif.com>
- [2] Espressif Systems. *ESP-IDF Programming Guide*. Disponible en: <https://docs.espressif.com/projects/esp-idf>
- [3] Raspberry Pi Foundation. *Raspberry Pi Documentation*. Disponible en: <https://www.raspberrypi.com/documentation>
- [4] Python Software Foundation. *Python Documentation*. Disponible en: <https://docs.python.org>


```
41         "embedded" : "external");
42
43     printf("Minimum free heap size: %d bytes\n",
44           esp_get_minimum_free_heap_size());
45
46     for (int i = 10; i >= 0; i--) {
47         printf("Restarting in %d seconds...\n", i);
48         vTaskDelay(1000 / portTICK_PERIOD_MS);
49     }
50
51     printf("Restarting now.\n");
52     fflush(stdout);
53     esp_restart();
54 }
```

Anexo B: Comandos utilizados

Listing 7: Comandos de verificación del entorno

```
1 python3 --version
2 pip3 --version
3 git --version
4 pip3 list
5 cat /etc/os-release
6 ls /dev/ttyACM*
7 idf.py set-target esp32s3
8 idf.py build
9 idf.py -p /dev/ttyACM0 flash monitor
```

Anexo C: Archivos incluidos

Reporte en PDF.

Captura del entorno Visual Studio Code + ESP-IDF.

Captura del monitor serial.

Captura de la Raspberry Pi configurada.

Código de prueba *Hello World*.